Relational Databases - Task Answers

1. What is normalisation?

Normalisation is the process of restructuring database tables to reduce data redundancy and eliminate data anomalies. The goal is to bring the database to a consistent state where each table represents a single subject, no data is unnecessarily duplicated across tables, and all attributes in a table are functionally dependent on the primary key. Normalisation works through a series of stages called normal forms (1NF, 2NF, 3NF), with each stage being structurally better than the previous one.

2. When is a table in 1NF (First Normal Form)?

A table is in First Normal Form (1NF) when:
- All of the key attributes are defined (a primary key exists),
- There are no repeating groups in the table (each cell contains a single value, not a list of values), and
- All attributes are dependent on the primary key.

In other words, data must be presented in a tabular format where each cell contains exactly one value.

3. When is a table in 2NF (Second Normal Form)?

A table is in Second Normal Form (2NF) when:
- It is already in 1NF, and
- It includes no partial dependencies -- that is, no non-key attribute is dependent on only a portion of the primary key.

2NF is only relevant when a table has a composite primary key (a key made up of two or more columns). If the primary key is a single column, a table in 1NF is automatically in 2NF.

4. When is a table in 3NF (Third Normal Form)?

A table is in Third Normal Form (3NF) when:
- It is already in 2NF, and
- It contains no transitive dependencies -- that is, no non-key attribute is dependent on another non-key attribute.

In 3NF, every non-key attribute must depend directly and only on the primary key, not on any other non-key column.

INVOICE TABLE ANALYSIS (Questions 5, 6, and 7)

The INVOICE table (provided):

INV_NUM	PROD_NUM	SALE_DATE	PROD_LABEL	VEND_CODE	VEND_NAME	QUANT_SOLD	PROD_PRICE
211347	AA-E3522QW	15-Jan-2018	Rot. sander	211	NeverFail	1	$34.46
211347	QD-300932X	15-Jan-2018	Drill bit	211	NeverFail	8	$2.73
211347	RU-995748G	15-Jan-2018	Band saw	309	BeGood	1	$31.59
211348	AA-E3522QW	15-Jan-2018	Rot. sander	211	NeverFail	2	$34.46
211349	GH-778345P	16-Jan-2018	Power drill	157	ToughGo	1	$69.32

Composite Primary Key: INV_NUM + PROD_NUM

---

5. Dependency Diagram and All Dependencies (1NF Analysis)

The table has no repeating groups and already contains single values per cell,
so it is already in 1NF. The composite primary key is INV_NUM + PROD_NUM.

Identified Dependencies:

  FULL dependency (on the full composite key):
    INV_NUM, PROD_NUM -> QUANT_SOLD
    (You need BOTH the invoice number AND product number to know the
     quantity sold on that specific invoice line.)

  PARTIAL dependencies (on only part of the composite key):
    INV_NUM -> SALE_DATE
    (The sale date belongs to the invoice, not the product.)

    PROD_NUM -> PROD_LABEL
    (The product label describes the product, not the invoice.)

    PROD_NUM -> VEND_CODE
    (The vendor code is a property of the product.)

    PROD_NUM -> PROD_PRICE
    (The product price belongs to the product, not the invoice.)

  TRANSITIVE dependency (non-key attribute depends on another non-key attribute):
    VEND_CODE -> VEND_NAME
    (The vendor name is determined by the vendor code, not directly
     by the primary key -- this is a transitive dependency.)

  Full dependency path (through partial):
    PROD_NUM -> VEND_CODE -> VEND_NAME

See file: Q5_dependency_diagram.drawio for the visual dependency diagram.

---
6. Remove Partial Dependencies -- Conversion to 2NF

To convert to 2NF, we remove all partial dependencies by splitting the table. Each part of the composite primary key becomes the primary key of a new table, and attributes that depend only on that part move into that table. Attributes that depend on the full composite key stay in a linking table.

Resulting 2NF tables:

```
  INVOICE (INV_NUM [PK], SALE_DATE)
    -- INV_NUM determines SALE_DATE (partial dep resolved)

  PRODUCT (PROD_NUM [PK], PROD_LABEL, VEND_CODE, VEND_NAME, PROD_PRICE)
    -- PROD_NUM determines all these attributes (partial deps resolved)
    -- Note: VEND_CODE -> VEND_NAME transitive dependency still exists here

  INVOICE_PRODUCT (INV_NUM [PK/FK], PROD_NUM [PK/FK], QUANT_SOLD)
    -- Full composite key determines QUANT_SOLD
```

Relational schemas:

```
  INVOICE (INV_NUM, SALE_DATE)
  PRODUCT (PROD_NUM, PROD_LABEL, VEND_CODE, VEND_NAME, PROD_PRICE)
  INVOICE_PRODUCT (INV_NUM, PROD_NUM, QUANT_SOLD)
```

See file: Q6_2NF_diagrams.drawio for the visual dependency diagrams.

---
7. Remove Transitive Dependencies -- Conversion to 3NF

The PRODUCT table from 2NF still has a transitive dependency:

```
  PROD_NUM -> VEND_CODE -> VEND_NAME
```

To convert to 3NF, we remove the transitive dependency by extracting VEND_CODE and VEND_NAME into their own table. VEND_CODE stays in PRODUCT as a foreign key to link to the new VENDOR table.

Resulting 3NF tables:

```
  INVOICE (INV_NUM [PK], SALE_DATE)

  PRODUCT (PROD_NUM [PK], PROD_LABEL, VEND_CODE [FK], PROD_PRICE)
    -- CHG_HOUR removed; VEND_CODE stays as foreign key to VENDOR

  VENDOR (VEND_CODE [PK], VEND_NAME)
    -- New table created from transitive dependency

  INVOICE_PRODUCT (INV_NUM [PK/FK], PROD_NUM [PK/FK], QUANT_SOLD)
```

Final relational schemas (3NF):
  INVOICE (INV_NUM, SALE_DATE)
  PRODUCT (PROD_NUM, PROD_LABEL, VEND_CODE, PROD_PRICE)
  VENDOR (VEND_CODE, VEND_NAME)
  INVOICE_PRODUCT (INV_NUM, PROD_NUM, QUANT_SOLD)

The database is now in 3NF: no partial dependencies, no transitive dependencies,
and each table represents a single subject.

See file: Q7_3NF_diagrams.drawio for the visual dependency diagrams.